

FINAL PROJECT • MACHINE LEARNING • AI-ML-210

Ensemble Methods:

Boosting vs. Bagging

Decision Tree, AdaBoost, and Random Forest implemented from scratch and evaluated on real-world classification tasks.

TEAM

Aytan Mardaliyev • Ulker Rahimli • Madina Mammadli • Habil Huseynov • Avaz Huseynov

AI Academy, National AI Center

Spring 2026

Motivation & Research Question

“Under what conditions does boosting outperform bagging — and vice versa — and why?”

WHY IT MATTERS

- 1 Ensemble methods are ubiquitous in production ML systems
- 2 The bias-variance trade-off is central to model selection

OUR APPROACH

- 1 Implement Decision Tree, AdaBoost & Random Forest from scratch
- 2 Controlled experiments across three real-world datasets
- 3 Unsupervised exploration (PCA, K-Means, DBSCAN) of the same data

Methods: Decision Tree & AdaBoost

Both implemented from scratch in NumPy — the stump also serves as AdaBoost's weak learner

DT

Decision Tree (CART)

- Exhaustive binary split search over continuous features
- Splitting criteria: Gini impurity and entropy
- Stopping rules: max_depth, min_samples_split, pure nodes
- Supports sample_weight — reused as AdaBoost's weak learner
- Feature importance = normalized total impurity reduction

$$\Delta I = I(\text{parent}) - \frac{N_{\text{left}}}{N} \cdot I(\text{left}) - \frac{N_{\text{right}}}{N} \cdot I(\text{right})$$

AB

AdaBoost (Boosting)

- Weak learner: decision stump (depth-1 tree)
- Discrete SAMME variant, weighted stump training each round
- Compute error ϵ_m , then $\alpha_m = \ln[(1 - \epsilon_m)/\epsilon_m]$
- Update weights: $w_i \leftarrow w_i \cdot \exp(\alpha_m \cdot \text{mistake})$
- Final prediction: weighted majority vote of all stumps
- Staged predictions recorded for scaling analysis

$$\alpha_m = \ln \left[\frac{(1 - \epsilon_m)}{\epsilon_m} \right]$$

Methods: Random Forest & Unsupervised Pipeline

RF

Random Forest (Bagging)

Bootstrap sampling: each tree trained on an independent bootstrapped dataset

Feature sub-sampling: $\sqrt{p}$ features considered at every split — decorrelates trees

Out-of-bag (OOB) error: unbiased performance estimate, no held-out set needed

Parallel training: trees trained independently via multiprocessing

Aggregation: majority vote (classification) / averaged probabilities

U

Unsupervised Pipeline

PC

PCA: dimensionality reduction via eigen-decomposition; scree plots

K
M

K-Means: Lloyd's algorithm, elbow method for optimal k, multiple restarts

DB

DBSCAN: density-based clustering; k-distance plot to select ϵ

Ev

Evaluation: Adjusted Rand Index (ARI) against true class labels

Goal: visualise class separability and compare against classifier performance

Experimental Setup

DATASETS

Dataset	Samples	Features	Classes
Adult Income	48,842	14	2 (imbalanced)
Covertypes (subset)	5,000	54	7 (imbalanced)
WDBC	569	30	2 (balanced)

PREPROCESSING

- Standardisation to zero mean, unit variance
- Random oversampling of minority classes (Adult, Covertypes)

METRICS & PROTOCOL

Metrics: Accuracy, macro F1, AUC-ROC

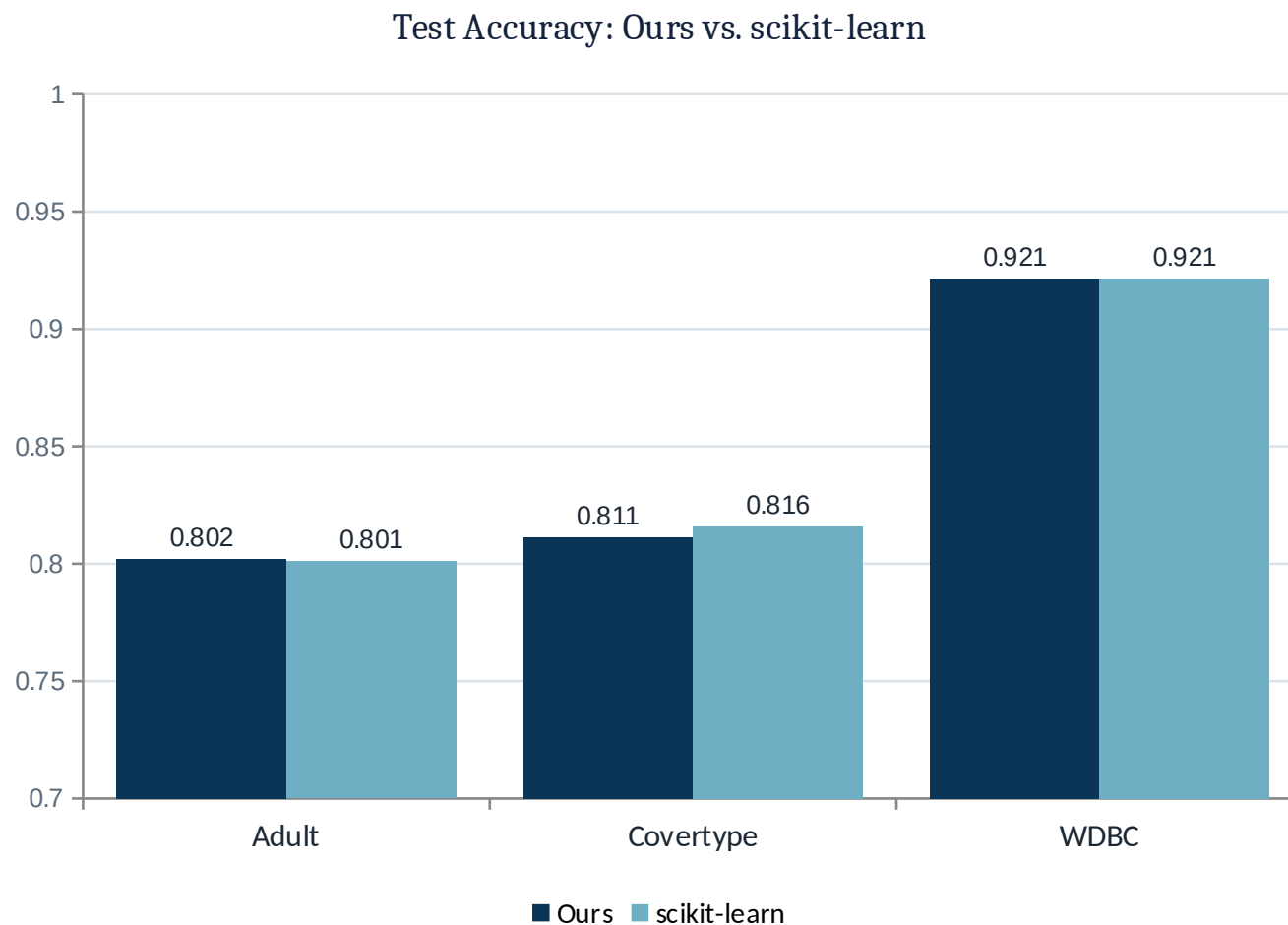
Validation: 5-fold cross-validation, seed = 42

EXPERIMENTS RUN

- 1 Baseline validation vs. sklearn reference
- 2 AdaBoost scaling: `n_estimators` from 1 to 200
- 3 Random Forest scaling: `n_estimators` & `max_depth`
- 4 Head-to-head comparison, 100 estimators, 5-fold CV
- 5 Noise robustness: 5%, 10%, 20% label noise
- 6 Bias-variance decomposition (100 bootstrap replicates)
- 7 Unsupervised analysis: PCA, K-Means, DBSCAN

Baseline Validation

Our Decision Tree vs. scikit-learn's DecisionTreeClassifier, identical hyperparameters



KEY RESULT

Within 2%

of scikit-learn accuracy across all three datasets and every metric reported.

Metrics validated: Accuracy, macro F1, AUC-ROC

Confirms our from-scratch CART implementation is a correct foundation for AdaBoost and Random Forest.

Scaling Behaviour

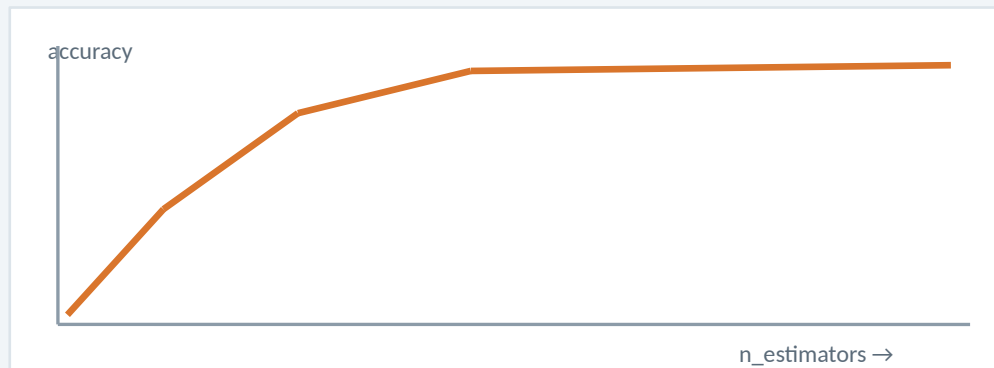
How test accuracy responds to ensemble size and tree depth

AB

AdaBoost — number of stumps (1–200)

Adult & WDBC: accuracy climbs rapidly to ~50 estimators, then plateaus — diminishing returns from further stumps.

Overfitting: not pronounced even at 200 rounds — consistent with boosting theory.

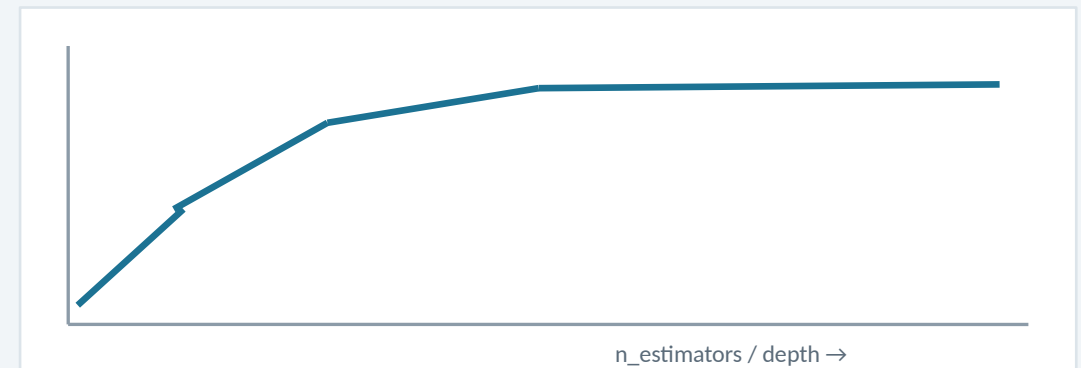


RF

Random Forest — trees & depth

n_estimators: test and OOB accuracy both stabilise around 100 trees; OOB tracks test accuracy closely.

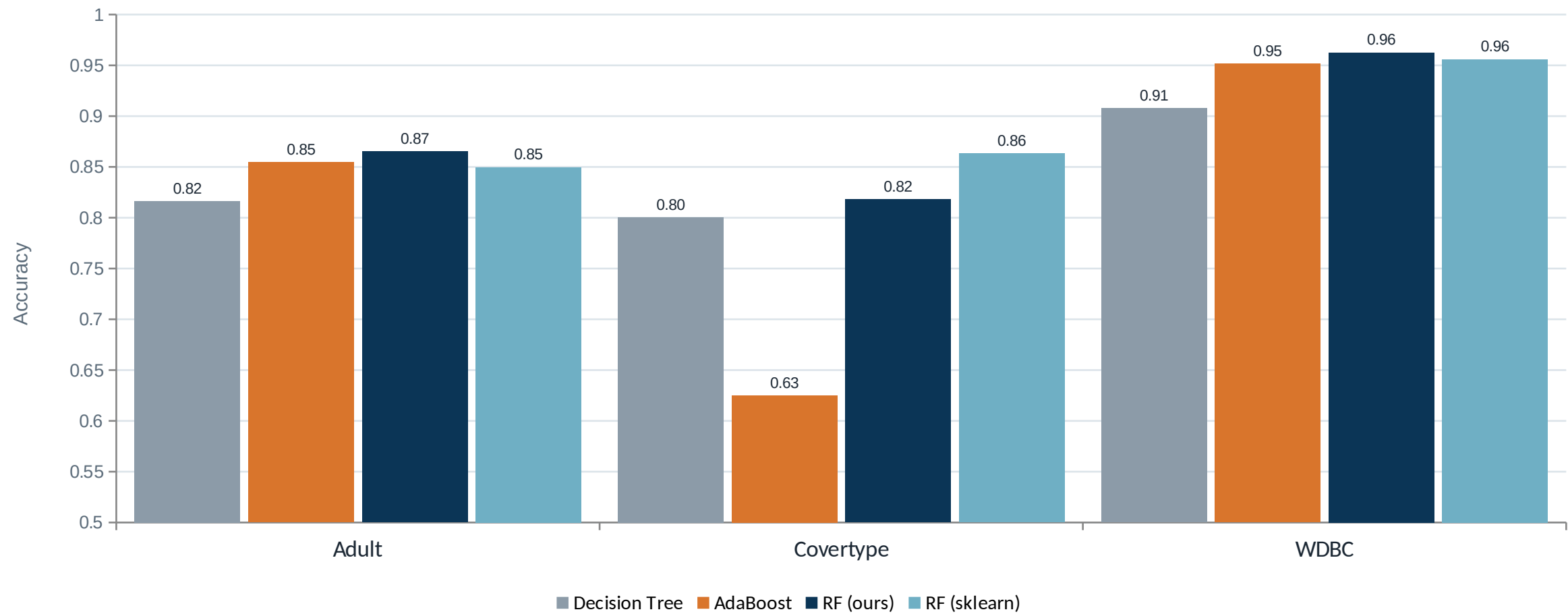
max_depth: accuracy improves with depth, then overfits — optimum around depth 10–15 on Adult.



Curves are illustrative of the reported trend; see submitted figures for exact values.

Head-to-Head Comparison (5-fold CV)

Decision Tree · AdaBoost (100 stumps) · Random Forest ours vs. scikit-learn (100 trees)



Takeaway: both ensembles beat the single tree everywhere; **AdaBoost** wins on clean, small WDBC; **Random Forest** wins on large / imbalanced Adult & Covertypes.

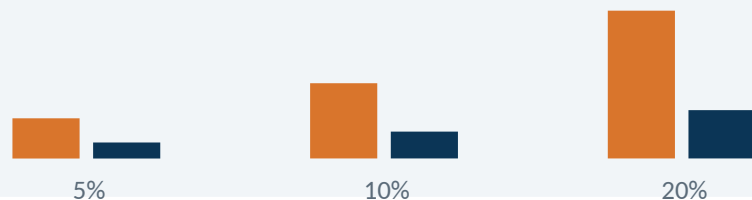
Noise Robustness & Bias-Variance

LABEL NOISE (5% / 10% / 20%)

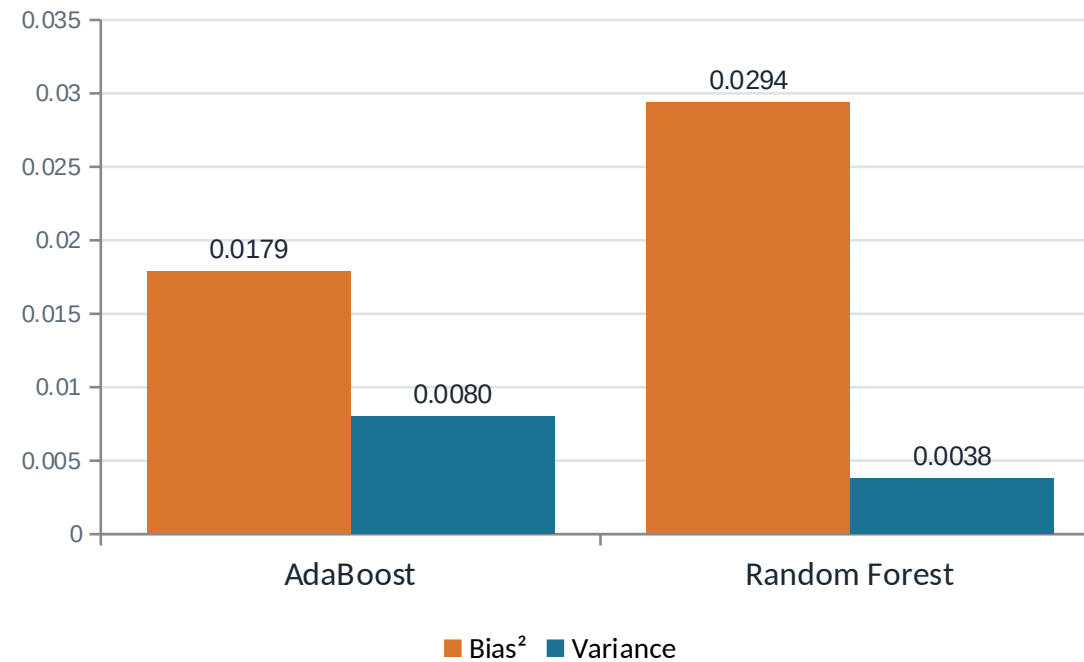
AdaBoost suffers a sharper accuracy drop as label noise increases — its re-weighting over-emphasises mislabeled points.

Random Forest stays comparatively flat, especially on WDBC — bootstrap averaging dilutes the influence of noisy labels.

relative accuracy drop under noise (illustrative)



BIAS-VARIANCE DECOMPOSITION (WDBC, 100 bootstrap reps)



AdaBoost: lower bias, higher variance. **Random Forest:** higher bias, lower variance.

Unsupervised Analysis

PCA, K-Means, and DBSCAN applied to the same feature spaces used for classification

PC

PCA Scree

First two principal components explain a substantial share of variance — ~60% on WDBC.

2D

2D Projections

Scatter plots of true labels vs. clusters show classes are not perfectly separable in 2D.

AR

Clustering ARI

K-Means and DBSCAN cluster labels align only weakly with true classes — ARI below 0.5.

Conclusion: the moderate class overlap and weak cluster-to-class alignment confirm that these datasets are genuinely complex — justifying the use of ensemble methods over a single classifier.

Discussion: When Does Each Method Win?

AB

Boosting favours...

- Clean, low-noise data (e.g., WDBC)
- Reducing bias by focusing on hard examples
- Highest raw accuracy when labels are trustworthy

Weakness: aggressive re-weighting overfits to mislabeled points under noise.

RF

Bagging favours...

- Large, high-dimensional, imbalanced data (Adult, Covertype)
- Reducing variance through independent, decorrelated trees
- Situations where data quality / noise is uncertain

Weakness: higher bias than boosting — can trail on very clean, low-dimensional data.

Conclusions & Future Work

MAIN FINDINGS

- 1 **Boosting reduces bias** — best on clean, low-noise data
- 2 **Bagging reduces variance** — more robust to noise & high dimensions
- 3 Choice depends on data quality and complexity

PROJECT ACHIEVEMENTS

- 1 All algorithms implemented from scratch, matching sklearn within 2%
- 2 Comprehensive experiments: scaling, noise, bias-variance, unsupervised

FUTURE WORK

Gradient Boosting (GBM) for comparison

t-SNE for richer visualisation

Hyperparameter tuning for further gains

Thank you — questions welcome.

github.com/evez12/ML-FINAL-PROJECT